

Matgeo Presentation - Problem 12.82

ee25btech11063 - Vejith

October 12, 2025

Question

let $\mathbf{M}$ be a 3×3 symmetric matrix with eigenvalues 1, 2 and 3. let $\mathbf{N}$ be any 3×3 matrix with real eigenvalues such that $\mathbf{MN} + \mathbf{NM} = 3\mathbf{I}$, where $\mathbf{I}$ is a 3×3 identity matrix. Then which of the following cannot be eigenvalue(s) of the matrix $\mathbf{N}$? (ST 2022)

a) $\frac{1}{4}$

b) $\frac{3}{4}$

c) $\frac{1}{2}$

d) $\frac{7}{4}$

Solution

let $\lambda_1, \lambda_2, \lambda_3$ be the eigenvalues of $\mathbf{N}$

By Eigenvalue decomposition

$$\mathbf{M} = \mathbf{Q}\mathbf{D}\mathbf{Q}^\top \quad (0.1)$$

where

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{Q}\mathbf{Q}^\top = \mathbf{I} \quad (0.2)$$

$$\mathbf{D} = \text{diag}(1, 2, 3) \quad (0.3)$$

$$\implies \mathbf{D} = \mathbf{Q}^\top \mathbf{M} \mathbf{Q} \quad (0.4)$$

$$\mathbf{M}\mathbf{N} + \mathbf{N}\mathbf{M} = 3\mathbf{I} \quad (0.5)$$

$$\mathbf{Q}^\top (\mathbf{M}\mathbf{N} + \mathbf{N}\mathbf{M}) \mathbf{Q} = \mathbf{Q}^\top 3\mathbf{I} \mathbf{Q} \quad (0.6)$$

$$\mathbf{Q}^\top \mathbf{M}\mathbf{N}\mathbf{Q} + \mathbf{Q}^\top \mathbf{N}\mathbf{M}\mathbf{Q} = 3\mathbf{I} \quad (0.7)$$

$$\implies \mathbf{Q}^\top \mathbf{M}\mathbf{N}\mathbf{Q} = \left(\mathbf{Q}^\top \mathbf{M}\mathbf{Q} \right) \left(\mathbf{Q}^\top \mathbf{N}\mathbf{Q} \right) \quad (0.8)$$

$$\implies \mathbf{Q}^\top \mathbf{N}\mathbf{M}\mathbf{Q} = \left(\mathbf{Q}^\top \mathbf{N}\mathbf{Q} \right) \left(\mathbf{Q}^\top \mathbf{M}\mathbf{Q} \right) \quad (0.9)$$

Solution

let

$$\mathbf{Q}^T \mathbf{N} \mathbf{Q} = \mathbf{N}' \quad (0.10)$$

$$\implies \mathbf{N} = \mathbf{Q} \mathbf{N}' \mathbf{Q}^T \quad (0.11)$$

$$\implies \mathbf{Q}^T \mathbf{M} \mathbf{N} \mathbf{Q} = \mathbf{D} \mathbf{N}' \quad (0.12)$$

$$\implies \mathbf{Q}^T \mathbf{N} \mathbf{M} \mathbf{Q} = \mathbf{N}' \mathbf{D} \quad (0.13)$$

From (0.11) and (0.12) (0.7) can be written as

$$\mathbf{D} \mathbf{N}' + \mathbf{N}' \mathbf{D} = 3\mathbf{I} \quad (0.14)$$

let $(\mathbf{D} \mathbf{N}')_{i,j}$ be the entry of i th column and j th row of the matrix $\mathbf{D} \mathbf{N}'$. As $\mathbf{D}$ is a diagonal matrix

$$(\mathbf{D} \mathbf{N}')_{i,j} = \mathbf{D}_i \mathbf{N}'_{i,j} \quad (0.15)$$

let $(\mathbf{N}' \mathbf{D})_{i,j}$ be the entry of i th column and j th row of the matrix $\mathbf{N}' \mathbf{D}$. As $\mathbf{D}$ is a diagonal matrix

Solution

$$(\mathbf{N}'\mathbf{D})_{i,j} = \mathbf{D}_j\mathbf{N}'_{i,j} \quad (0.16)$$

$$\implies (\mathbf{D}_i + \mathbf{D}_j) \mathbf{N}'_{i,j} = 3(\mathbf{I})_{i,j} \quad (0.17)$$

for $i \neq j$

$$(\mathbf{D}_i + \mathbf{D}_j) \mathbf{N}'_{i,j} = 0 \quad (0.18)$$

$$\implies \mathbf{N}'_{i,j} = 0 \quad (0.19)$$

for $i = j$

$$(\mathbf{D}_i + \mathbf{D}_j) \mathbf{N}'_{i,j} = 3 \quad (0.20)$$

$$\implies \mathbf{N}'_{i,j} = \frac{3}{2D_i} \quad (0.21)$$

$\implies$ The diagonal values of $\mathbf{N}'$ are $\frac{3}{2}$ and $\frac{3}{4}$ and $\frac{1}{2}$
In (0.11) $\mathbf{Q}$ is an orthogonal matrix and $\mathbf{N}'$ is a diagonal matrix so we can say that (11) is the eigenvalue decomposition of $\mathbf{N}$

$$\implies \mathbf{N} = \mathbf{Q}\mathbf{N}'\mathbf{Q}^\top$$

Solution

$$\mathbf{N}' = \text{diag}(\lambda_1, \lambda_2, \lambda_3) \quad (0.22)$$

$$\implies \lambda_1 = \frac{3}{2} \text{ and } \lambda_2 = \frac{3}{4} \text{ and } \lambda_3 = \frac{1}{2} \quad (0.23)$$

C Code: eigen.c

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define N 3
#define MAX_ITERS 100
#define EPS 1e-12

void max_offdiag(double A[N][N], int *pi, int *pj) {
    double max = 0.0;
    int imax = 0, jmax = 1;
    for (int i = 0; i < N; ++i) {
        for (int j = i+1; j < N; ++j) {
            double v = fabs(A[i][j]);
            if (v > max) {
                max = v;
                imax = i; jmax = j;
            }
        }
    }
    *pi = imax; *pj = jmax;
}

void jacobi_eigensolver(double A[N][N], double eig[N]) {
    double V[N][N] = {0}; // eigenvector matrix (not used for result but kept)
    for (int i=0; i<N; i++) V[i][i]=1.0;

    double B[N][N];
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            B[i][j]=A[i][j];
    for (int iter = 0; iter < MAX_ITERS; ++iter) {
        int p,q;
        max_offdiag(B, &p, &q);
```

C Code: eigen.c

```
double bpq = B[p][q];
if (fabs(bpq) < EPS) break;

double app = B[p][p];
double aqq = B[q][q];

double tau = (aqq - app) / (2.0 * bpq);
double t;
if (tau >= 0.0) t = 1.0 / (tau + sqrt(1.0 + tau*tau));
else t = -1.0 / (-tau + sqrt(1.0 + tau*tau));
double c = 1.0 / sqrt(1.0 + t*t);
double s = t * c;
double tauPrime = s/(1.0 + c);

// update matrix B
double app_new = app - t * bpq;
double aqq_new = aqq + t * bpq;
B[p][p] = app_new;
B[q][q] = aqq_new;
B[p][q] = B[q][p] = 0.0;

for (int k = 0; k < N; ++k) {
    if (k != p && k != q) {
        double bkp = B[k][p];
        double bkq = B[k][q];
        B[k][p] = B[p][k] = bkp - s*(bkq + tauPrime*bkp);
        B[k][q] = B[q][k] = bkq + s*(bkp - tauPrime*bkq);
    }
}

for (int k = 0; k < N; ++k) {
    double vkp = V[k][p];
    double vkq = V[k][q];
```


C Code: eigen.c

```
        V[k][p] = vkp - s*(vkq + tauPrime*vkp);
        V[k][q] = vkq + s*(vkp - tauPrime*vkq);
    }}
    for (int i = 0; i < N; ++i) eig[i] = B[i][i];
}

int main(void) {

    double M[N][N] = {
        {1.0, 0.0, 0.0},
        {0.0, 2.0, 0.0},
        {0.0, 0.0, 3.0}
    };
    double lambda[N];
    jacobi_eigensolver(M, lambda);

    double mu[N];
    for (int i = 0; i < N; ++i) {
        if (fabs(lambda[i]) < 1e-14) {
            fprintf(stderr, "Eigenvalue near zero; cannot compute mu = 3/(2*lambda).\n");
            return 1;
        }
        mu[i] = 3.0 / (2.0 * lambda[i]);
    }

    FILE *fp = fopen("eigen.dat", "w");
    if (!fp) {
        perror("fopen");
        return 1;
    }
    for (int i = 0; i < N; ++i) {
        fprintf(fp, "%.12g\n", mu[i]);
    }
    fclose(fp);
    printf("Wrote 3 eigenvalues of N to eigen.dat\n");
    return 0;
}
```

Python: solution.py

```
import numpy as np

# Step 1: Define symmetric matrix M (diagonal with eigenvalues 1,2,3)
M = np.diag([1.0, 2.0, 3.0])

# Step 2: Eigen decomposition of M
eigvals_M, Q = np.linalg.eigh(M)

# Step 3: Compute  $N' = \text{diag}(3 / (2 * \_i))$ 
eigvals_N = 3.0 / (2.0 * eigvals_M)
N_dash = np.diag(eigvals_N)

# Step 4: Reconstruct  $N = Q * N' * Q^T$ 
N = Q @ N_dash @ Q.T

# Step 5: Verification:  $M*N + N*M$ 
verify = M @ N + N @ M

# Step 6: Display results
print("Matrix_M:")
print(M)

print("\nEigenvalues_of_M:")
print(eigvals_M)

print("\nEigenvalues_of_N:")
print(eigvals_N)

print("\nMatrix_N:")
print(N)

print("\nVerification_(M*N+_N*M):")
print(verify)
```